编码规范

一、总则

1.1 目的

本规范旨在统一企业内 Python 代码风格，提升代码可读性、可维护性与团队协作效率，降低长期维护成本。所有 Python 项目均须遵循本规范，特殊场景可在项目内另行约定补充细则。

1.2 基本原则

- **可读性优先**：代码被阅读的次数远多于编写次数，优先保证他人易于理解
- **一致性优先**：项目内保持风格统一比严格遵守单条规则更重要
- **渐进式落地**：存量代码可逐步整改，新增代码必须符合规范
- **工具化落地**：尽可能通过自动化工具执行检查，减少人工评审成本

1.3 参考标准

- 以 **PEP 8** 为基础标准
- 参考 **Google Python Style Guide** 补充工程实践约定
- 采用 **Black** 格式化规则（行宽 88 字符）作为事实格式化标准

二、代码格式规范

2.1 缩进与换行

- **强制**：使用 4 个空格缩进，禁止使用 Tab，禁止混用 Tab 与空格
- **强制**：多行续行使用悬挂缩进，第一行不留参数，续行额外缩进 4 空格

```
# 推荐
def create_user(
    username: str,
    email: str,
    role: str = "user",
) -> User:
    pass

# 不推荐
def create_user(username: str, email: str,
                role: str = "user") -> User:
    pass
```

- **推荐**：长表达式在二元运算符**之前**换行，便于阅读逻辑链

```
total = (base_price
         + tax_amount
         - discount
         + shipping_fee)
```

2.2 行宽与空行

- **强制**：单行最大长度 **88 字符**（兼容 Black 默认配置）
- **强制**：顶层函数与类定义之间空 **2 行**
- **强制**：类内方法定义之间空 **1 行**
- **推荐**：逻辑块之间可使用 1 个空行分隔，避免连续多个空行
- **强制**：文件末尾保留且仅保留 1 个空行

2.3 空格使用

- **强制**：二元运算符（=、==、<、+、and 等）两侧各留 1 个空格
- **强制**：函数默认参数的 = 两侧**不加空格**
- **强制**：逗号、冒号、分号前不加空格，后加 1 个空格（行尾除外）
- **强制**：括号内侧不加空格
- **禁止**：为对齐赋值语句而人为添加多余空格

```
# 推荐
x = 1
y = 2
long_variable = 3

# 不推荐
x             = 1
y             = 2
long_variable = 3
```

2.4 字符串

- **强制**：项目内统一使用单引号或双引号，推荐使用双引号
- **强制**：多行字符串使用三双引号 `"""..."""`
- **推荐**：字符串拼接优先使用 f-string，其次是 `str.format()`，避免 + 拼接大量字符串
- **禁止**：使用隐式字符串拼接（易出 bug），显式使用 + 或 f-string

三、命名规范

3.1 命名总原则

- 命名应当**语义清晰、见名知意**，避免无意义缩写
- 避免使用单字母变量名（循环计数器 i/j/k、数学公式变量除外）
- 避免使用与内置函数/关键字重名的名称，必要时加后缀下划线（如 `class_`）

3.2 具体命名约定

元素类型	命名风格	示例
模块/包名	蛇形命名（全小写+下划线）	`user_service.py`, `data_processing`
类名	大驼峰（帕斯卡命名）	`UserService`, `DataProcessor`
异常名	大驼峰，以 `Error` 结尾	`ConfigNotFoundError`
函数/方法名	蛇形命名	`get_user_info`, `calculate_total`
变量名	蛇形命名	`user_name`, `order_list`
常量	全大写+下划线	`MAX_RETRY_COUNT`, `DEFAULT_TIMEOUT`
私有属性/方法	单下划线前缀	`_internal_cache`, `_validate_params`

保护属性	单下划线前缀	`_protected_field`
类方法/静态方法	蛇形命名	`@classmethod def from_dict(cls, data):`

3.3 下划线使用约定

- **单前导下划线 `_var`**：表示模块/类内部使用，弱私有约定，`from module import *` 不会导入
- **双前导下划线 `__var`**：触发名称修饰，用于避免子类属性名冲突，**非必要不使用**
- **前后双下划线 `__init__`**：Python 魔法方法，仅使用系统定义，禁止自定义
- **单后缀下划线 `var_`**：用于避免与 Python 关键字冲突，如 `type_`, `class_`

四、导入规范

4.1 导入顺序

强制按以下顺序分组，组间空 1 行：

1. 标准库导入
2. 第三方库导入
3. 本地项目/应用导入

```
import os
import sys
from datetime import datetime

import requests
import pandas as pd

from myapp.models import User
from myapp.utils import logger
```

4.2 导入规则

- **强制**：每个 import 独占一行，禁止 `import os, sys`
- **强制**：禁止使用 `from module import *` 通配符导入
- **强制**：禁止相对导入（除包内测试场景外），使用绝对导入
- **推荐**：导入路径从项目根目录开始，避免深层相对路径
- **禁止**：循环导入，出现时应重构公共代码到独立模块

4.3 导入别名

- 通用库可使用约定俗成的别名：`import pandas as pd`, `import numpy as np`
- 其他库不建议随意起别名，避免增加阅读成本

五、注释与文档字符串

5.1 注释原则

- 注释解释**为什么这么做**，而非复述代码做了什么
- 代码变更时同步更新注释，过期注释比无注释更有害

- 禁止注释掉的代码提交到仓库，应直接删除

5.2 注释格式

- **块注释**：置于代码块上方，与代码同缩进，`#` 后空 1 格
- **行内注释**：与代码至少间隔 2 个空格，`#` 后空 1 格，避免滥用
- **TODO 注释**：格式为 `# TODO(责任人): 描述内容`，必须标注责任人

5.3 文档字符串（Docstring）

强制所有公共模块、类、函数、方法必须编写文档字符串。

采用 **Google 风格** docstring：

```
def fetch_user(user_id: int, include_deleted: bool = False) -> Optional[User]:
    """根据用户 ID 获取用户信息。

    Args:
        user_id: 用户唯一标识 ID
        include_deleted: 是否包含已删除用户，默认为 False

    Returns:
        用户对象，未找到时返回 None

    Raises:
        ValueError: user_id 小于等于 0 时抛出
        DatabaseError: 数据库连接异常时抛出

    Examples:
        >>> user = fetch_user(1001)
        >>> print(user.name)
    """
    pass
```

- 模块级 docstring 说明模块功能、主要导出类/函数
- 类 docstring 说明类的职责、主要属性与典型用法
- 私有方法可不写完整 docstring，但复杂逻辑仍需注释说明

六、类型注解

6.1 基本要求

- **强制**：所有公共 API 的函数参数与返回值必须添加类型注解
- **推荐**：复杂项目全量启用类型注解，配合 mypy 静态检查
- Python 版本 ≥ 3.9 优先使用内置泛型（`list[str]` 而非 `List[str]`）

6.2 常用类型约定

- 可选值使用 `Optional[T]` 或 `T | None`（Python 3.10+）
- 无返回值使用 `-> None`
- 字典结构复杂时使用 `TypedDict` 或 Pydantic 模型，避免 `dict[str, Any]`
- 回调函数使用 `Callable[[ArgType1, ArgType2], ReturnType]`

6.3 类型检查

- CI 流水线集成 mypy 检查，错误级别可按项目逐步收紧
- 第三方库缺失类型存根时，使用 `# type: ignore` 并标注原因

七、异常处理规范

7.1 异常捕获原则

- **禁止**：裸 `except:` 捕获所有异常，必须指定具体异常类型
- **禁止**：捕获异常后静默吞掉（空 except 块），至少记录日志
- **推荐**：尽量缩小 try 块范围，只包裹可能抛出异常的代码
- **推荐**：使用 `finally` 执行资源清理，或使用上下文管理器

```
# 推荐
try:
    result = risky_operation()
except (ConnectionError, TimeoutError) as e:
    logger.error(f"操作失败: {e}")
    raise BusinessError("服务暂时不可用") from e
```

7.2 自定义异常

- 业务异常须继承自基础业务异常类，形成异常层级
- 异常命名以 `Error` 结尾，语义清晰
- 抛出异常时携带足够的上下文信息，便于排查问题

7.3 异常抛出

- 参数校验失败抛出 `ValueError` / `TypeError`
- 资源不存在抛出 `FileNotFoundError` / 自定义 NotFound 异常
- 使用 `raise ... from ...` 保留原始异常栈，便于调试

八、函数与类设计

8.1 函数设计

- **单一职责**：一个函数只做一件事，函数名能准确描述其行为
- **参数控制**：参数不超过 5 个，过多时使用数据类或配置对象封装
- **避免副作用**：函数尽量纯净化，修改输入参数时必须在 docstring 中明确说明
- **默认参数**：禁止使用可变对象作为默认参数（如 `[]`, `{}`）

```
# 错误示范
def add_item(item, lst=[]):
    lst.append(item)
    return lst

# 正确示范
def add_item(item, lst=None):
```

```
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

8.2 类设计

- 优先使用组合而非继承，继承层级不超过 3 层
- 数据容器优先使用 `@dataclass` 或 Pydantic 模型
- 类属性明确区分类变量与实例变量
- 公开接口最小化，内部实现使用单下划线前缀标记

8.3 模块设计

- 模块保持合理大小，单文件建议不超过 500 行
- 避免循环依赖，出现时抽离公共逻辑
- 模块顶层只做定义，不执行耗时操作（数据库连接、网络请求等）

九、性能与安全规范

9.1 性能实践

- 避免在循环中进行重复计算或重复查询
- 大数据量处理使用生成器而非列表，节省内存
- 字符串拼接优先 f-string，其次 `str.join()`，避免循环中 `+=`
- 合理使用缓存装饰器（`functools.lru_cache`），注意内存泄漏风险

9.2 安全规范

- **禁止**：代码中硬编码密钥、密码、Token 等敏感信息，使用环境变量或配置中心
- **禁止**：使用 `eval()`、`exec()` 执行不可信输入
- **禁止**：SQL 语句使用字符串拼接，必须使用参数化查询
- 文件路径操作校验输入，防止路径遍历攻击
- 外部输入必须做合法性校验，禁止信任前端传入数据

十、测试规范

10.1 测试基本原则

- 核心业务逻辑必须编写单元测试
- 公共 API 必须有测试覆盖
- 测试代码同样遵守本编码规范

10.2 测试约定

- 测试文件以 `test_` 前缀命名，置于 `tests/` 目录
- 测试函数以 `test_` 前缀命名

- 每个测试用例只验证一个场景，用例之间保持独立
- 使用 arrange-act-assert 三段式结构组织测试
- 禁止在测试中使用 `print` 输出，使用断言验证结果

十一、标准项目目录结构

```
project_name/
├── src/                          # 源码根目录
│   └── myapp/                    # 主包
│       ├── __init__.py
│       ├── api/                  # 接口层
│       ├── service/              # 业务逻辑层
│       ├── dao/                  # 数据访问层
│       ├── models/               # 数据模型
│       ├── common/               # 公共组件
│       │   ├── exceptions.py     # 自定义异常
│       │   ├── constants.py      # 常量定义
│       │   └── utils.py          # 工具函数
│       └── config.py             # 配置管理
├── tests/                        # 测试目录
│   ├── unit/                     # 单元测试
│   └── integration/              # 集成测试
├── scripts/                      # 运维脚本
├── configs/                      # 配置文件
├── docs/                         # 项目文档
├── pyproject.toml                # 项目配置（格式化、lint、依赖）
├── requirements.txt              # 或 poetry/pipenv 管理
├── README.md
└── .gitignore
```

十二、工具链配置

12.1 必装工具

工具	用途	配置建议
Black	代码自动格式化	行宽 88，默认配置
isort	导入排序	兼容 Black 模式
flake8	代码风格检查	忽略 E501（行宽由 Black 控制）
mypy	静态类型检查	渐进式启用
pylint	代码质量检查	可按项目调整规则等级

12.2 pyproject.toml 参考配置

```
[tool.black]
line-length = 88
target-version = ['py310']

[tool.isort]
profile = "black"
line_length = 88
```

```
[tool.flake8]
max-line-length = 88
extend-ignore = ["E203", "W503"]

[tool.mypy]
python_version = "3.10"
warn_return_any = true
warn_unused_configs = true
disallow_untyped_defs = false
```

12.3 CI 集成

- 提交代码前本地执行格式化与 lint 检查
- CI 流水线强制运行：单元测试 + 代码风格检查 + 类型检查
- 不合规代码禁止合入主干分支

十三、版本控制约定

- 提交信息遵循约定式提交（Conventional Commits）：`feat:`, `fix:`, `refactor:`, `docs:` 等
- 每次提交聚焦单一功能点，禁止大杂烩提交
- 合入前必须经过代码评审，评审重点关注业务逻辑正确性、安全性与可维护性
